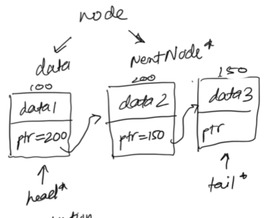
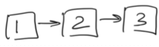
Linked list



Implementation either ctl or scratch



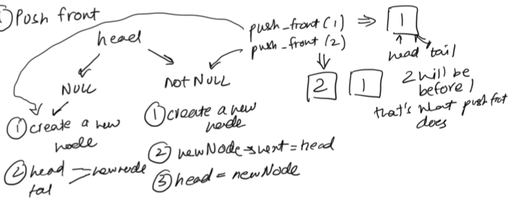

```
class Node{
public:
    int data;
    Node* next;
    Node(int val){
        data = val;
        next = NULL;
    }
};
```

← this will just make each node but to connect and make LL we need a new class

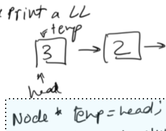
```
class List{
    Node * head;
    Node * tail;
public:
    List(){
        head = tail = NULL;
};};
```

we have
push_front, pop_front
push_back, pop_back

① Push front

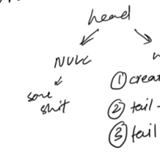


* Print a LL




```
Node * temp = head;
while (temp != NULL){
    cout << temp->data << " ";
    temp = temp->next;
}
```

② Push back

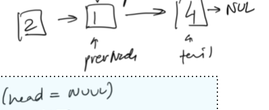


③ Pop front

delete first

```
Check if empty first {
    ...
}
Node *temp = head
head = head->next
temp->next = NULL
delete temp;
```

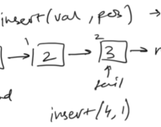
④ Pop back



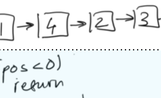

```
if (head = NULL)
    return
Node *temp = head
while (temp->next != tail){
    temp = temp->next
}
temp->next = NULL
delete tail
tail = temp
```

Insert in the middle of LL

insert(val, pos) → ctl



insert(4, 1)




```
if (pos<0)
    return
if (pos==0)
    push_front(val)
Node *temp = head
for (i=0 to i<pos-1)
    temp = temp->next
newNode->next = temp->next
temp->next = newNode
```

Search

Search(key)

create temp then compare with key also keep index then when reached then return